

Project: Investment Portfolio Management System

1. Overview

The **Investment Portfolio Management System** is a web-based application that enables financial institutions and individual investors to manage and monitor their investment portfolios. It provides functionality for portfolio creation, asset allocation, risk assessment, performance tracking, and reporting.

This system is designed using the **MVC architecture**, making it compatible with both **Java (Spring MVC)** and **.NET (ASP.NET Core MVC)** frameworks.

2. Core Modules Description

The system is divided into five primary modules, each addressing a critical aspect of investment portfolio management.

2.1 User Management

This module handles user account creation, role assignment (Investor, Admin), and profile management.

2.2 Portfolio Management

This module allows users to create and manage investment portfolios, including adding and modifying assets like stocks, bonds, and mutual funds.

2.3 Risk Assessment

This module calculates the risk level of portfolios based on asset allocation, market trends, and user-defined parameters.

2.4 Performance Tracking

This module monitors the performance of assets and portfolios, providing historical and real-time data.

2.5 Reporting and Analytics

This module generates reports and analytics for users and administrators to make informed decisions.

3. Module-Level Design

3.1 User Management Module

Purpose: Handles user registration, authentication, and profile management.

- **Controller:**
 - UserController
 - registerUser()
 - loginUser()

- updateUserProfile()
- **Service:**
 - UserService
 - Validates user details and manages authentication.
- **Model:**
 - **User Entity**
 - Attributes:
 - userId (PK)
 - username
 - password
 - email
 - role (Investor, Admin)

3.2 Portfolio Management Module

Purpose: Enables users to manage investment portfolios.

- **Controller:**
 - PortfolioController
 - createPortfolio()
 - addAssetToPortfolio()
 - viewPortfolioDetails()
- **Service:**
 - PortfolioService
 - Handles portfolio creation and asset allocation logic.
- **Model:**
 - **Portfolio Entity**
 - Attributes:
 - portfolioId (PK)
 - userId (FK)
 - portfolioName
 - creationDate
 - **Asset Entity**
 - Attributes:
 - assetId (PK)
 - portfolioId (FK)
 - assetType (Stock, Bond, Mutual Fund)
 - assetName
 - quantity
 - purchasePrice

3.3 Risk Assessment Module

Purpose: Assesses the risk level of portfolios based on asset allocation.

- **Controller:**
 - RiskController
 - calculateRisk()
 - viewRiskAnalysis()

- **Service:**
 - RiskService
 - Implements algorithms for risk calculation.
- **Model:**
 - **Risk Entity**
 - Attributes:
 - riskId (PK)
 - portfolioId (FK)
 - riskLevel (Low, Medium, High)
 - analysisDate

3.4 Performance Tracking Module

Purpose: Tracks the performance of assets and portfolios.

- **Controller:**
 - PerformanceController
 - getAssetPerformance()
 - getPortfolioPerformance()
- **Service:**
 - PerformanceService
 - Aggregates data from market APIs for performance tracking.
- **Model:**
 - **Performance Entity**
 - Attributes:
 - performanceId (PK)
 - assetId (FK)
 - currentValue
 - profitLoss
 - lastUpdated

3.5 Reporting and Analytics Module

Purpose: Provides insights into portfolio performance and risk.

- **Controller:**
 - ReportController
 - generatePortfolioReport()
 - generateRiskReport()
 - generateAssetAnalysisReport()
- **Service:**
 - ReportService
 - Generates detailed reports and analytics.
- **Model:**
 - **Report Entity**
 - Attributes:
 - reportId (PK)
 - reportType (Portfolio, Risk, Asset)
 - generatedDate

4. Database Schema

4.1 Table Definitions

1. User Table

```
CREATE TABLE User (  
    userId INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(100),  
    password VARCHAR(100),  
    email VARCHAR(100),  
    role ENUM('Investor', 'Admin')  
);
```

2. Portfolio Table

```
CREATE TABLE Portfolio (  
    portfolioId INT AUTO_INCREMENT PRIMARY KEY,  
    userId INT,  
    portfolioName VARCHAR(100),  
    creationDate DATE,  
    FOREIGN KEY (userId) REFERENCES User(userId)  
);
```

3. Asset Table

```
CREATE TABLE Asset (  
    assetId INT AUTO_INCREMENT PRIMARY KEY,  
    portfolioId INT,  
    assetType ENUM('Stock', 'Bond', 'Mutual Fund'),  
    assetName VARCHAR(100),  
    quantity INT,  
    purchasePrice DECIMAL(10, 2),  
    FOREIGN KEY (portfolioId) REFERENCES Portfolio(portfolioId)  
);
```

4. Risk Table

```
CREATE TABLE Risk (  
    riskId INT AUTO_INCREMENT PRIMARY KEY,  
    portfolioId INT,  
    riskLevel ENUM('Low', 'Medium', 'High'),  
    analysisDate DATE,  
    FOREIGN KEY (portfolioId) REFERENCES Portfolio(portfolioId)  
);
```

5. Performance Table

```
CREATE TABLE Performance (  
    performanceId INT AUTO_INCREMENT PRIMARY KEY,  
    assetId INT,
```

```
currentValue DECIMAL(10, 2),
profitLoss DECIMAL(10, 2),
lastUpdated DATE,
FOREIGN KEY (assetId) REFERENCES Asset(assetId)
);
```

6. Report Table

```
CREATE TABLE Report (
    reportId INT AUTO_INCREMENT PRIMARY KEY,
    reportType ENUM('Portfolio', 'Risk', 'Asset'),
    generatedDate DATE
);
```

5. Local Deployment Details

1. Environment Setup:

- Install MySQL or SQL Server for database management.
- Install Java (JDK) or .NET SDK (ASP.NET Core).
- Configure database connections in the application's configuration file.

2. Deployment Steps:

- Clone the repository.
- Build the project using Maven (Java) or Visual Studio (ASP.NET Core).
- Run the application server.
- Access the application at <http://localhost:8080> (Java) or <http://localhost:5000> (.NET).

6. Conclusion

The **Investment Portfolio Management System** simplifies the complex process of managing investment portfolios for individuals and financial institutions. Its modular design ensures scalability and adaptability for future enhancements.